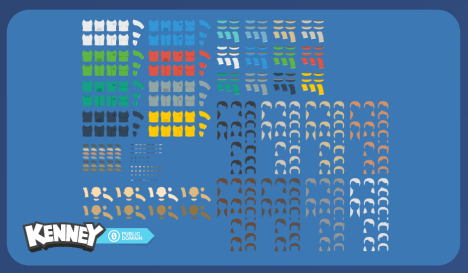
SOFTWARE-ONTWERP · INLEVERDOCUMENT

Software-ontwerp ZorgApp 1.3

Ontwerpdocumentatie van de Unity-game, de .NET Web API, de koppeling met Microsoft SQL Server en de CI/CD-pipeline. Deze versie is opgemaakt als definitieve inlever-PDF, inclusief voorblad, inhoudsopgave, paginanummers, visuele diagrammen en technische onderbouwing.



Frontend
Unity 6.0.36f1

Backend
ASP.NET Core Web API (.NET 9)

Opslag
Microsoft SQL Server + Dapper

Deployment
GitHub Actions → Azure App Service

Projectgegevens

Groepsnaam: BitByBit

Klas: 2C (herkansing)

Documenttype: Software-ontwerp

Teamleden

- Yazan Dawud
- Rishyan Syal
- Fabiano Pungol

Inhoudsopgave

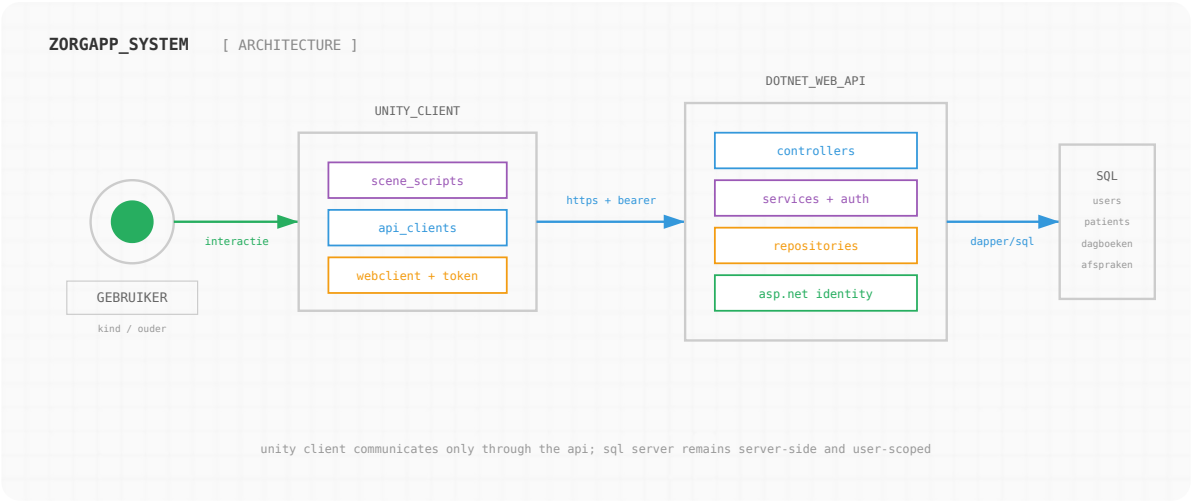
1. Inleiding 3
2. Architectuurprincipe 3
3. Frontendanalyse: Unity-game 4
4. Backendanalyse: .NET Web API 5
5. Relatie tussen API en Microsoft SQL Server 6
6. Belangrijkste runtime-interacties 6
7. CI/CD-pipeline 7
8. Sterke punten, risico's en alternatieven 8
9. Koppeling aan de rubric 8
10. Conclusie 8

Wat in dit document zit

- Architectuuroverzicht van het volledige systeem
- Analyse van Unity, Web API en SQL Server
- UML class diagram
- ERD van het datamodel
- CI/CD-diagram van de GitHub Actions workflow
- Concrete ontwerpkeuzes, risico's en verbeterpunten

Doel van de documentatie

Deze documentatie is bedoeld om te laten zien hoe de applicatie technisch is opgebouwd, welke componenten met elkaar samenwerken en waarom de gekozen architectuur logisch is voor onderhoudbaarheid, schaalbaarheid, security en performance.



Preview van het architectuurdiagram dat verderop in het document volledig wordt toegelicht.

1. Inleiding

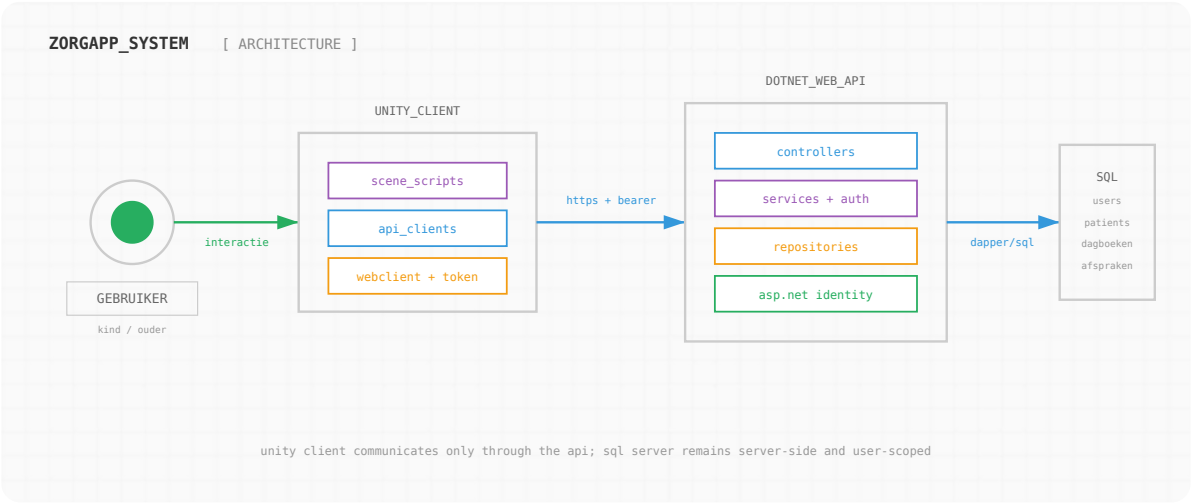
De ZorgApp bestaat uit twee repositories die samen één functioneel systeem vormen. De repository **avansbitbybit/zorgapp1.3** bevat de Unity-client, waarin de gebruiker kan inloggen, door scènes navigeren, een avatar aanpassen, dagboekitems bekijken en afspraken beheren. De repository **avansbitbybit/zorgapp1.3webapi** bevat de server-side logica: authenticatie, autorisatie, businessregels en databasecommunicatie.

Dit ontwerpdocument is gebaseerd op de werkelijke implementatie van beide codebases. Daarbij zijn de scene-configuratie, API-clients, repositories, modellen, testprojecten en de GitHub Actions workflow meegenomen om een volledig beeld van het systeem te geven.

2. Architectuurprincipe

Het systeem volgt primair een **client-serverarchitectuur**. De Unity-app is verantwoordelijk voor presentatie, gebruikersinteractie en scènebesturing. De Web API is verantwoordelijk voor authenticatie, autorisatie, gegevensverwerking en persistente opslag. Binnen de backend is daarnaast een **gelaagde architectuur** zichtbaar: controllers, services, repositories en modellen hebben elk een eigen verantwoordelijkheid.

- **Client-server:** Unity communiceert via HTTP met de API.
- **Separation of concerns:** UI, transport, businesslogica en persistence zijn gescheiden.
- **User-scoped design:** data wordt server-side gefilterd op `UserId`.
- **Deployable backend:** de API kan via een pipeline automatisch worden gebouwd, getest en gedeployed.



Figuur 1 — Hoofdarchitectuur van het systeem. De Unity-client praat uitsluitend met de API; SQL Server blijft volledig server-side.

> **Belangrijk ontwerpprincipe:** de Unity-client praat nooit direct met SQL Server. Alle toegang loopt via de Web API. Daardoor blijven beveiliging, validatie en gebruikersafscherming gecentraliseerd op de server.

3. Frontendanalyse: Unity-game

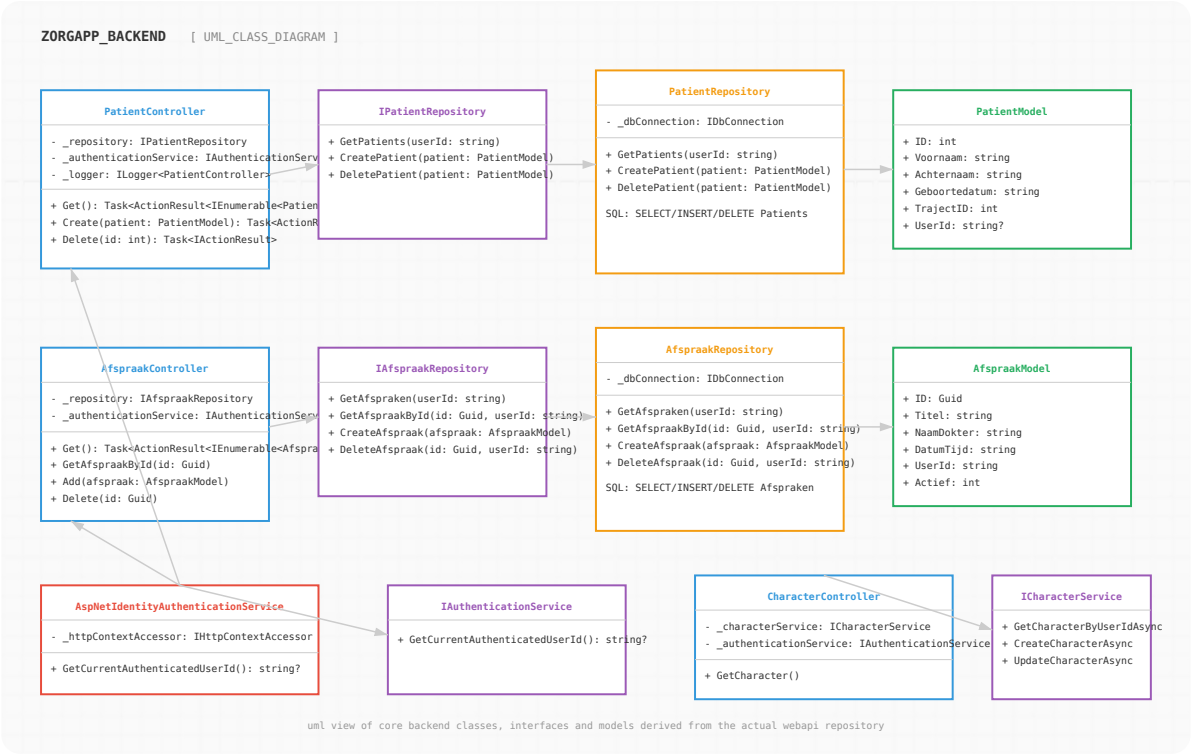
De Unity-repo draait op **Unity 6.0.36f1** en bevat 21 actieve scènes. In `EditorBuildSettings.asset` zijn onder andere *IntroductieScene*, *LoginScene*, *Behandeltraject*, *DagboekScene*, *Accountgegevens*, *MenuScene*, *Charactercreator* en meerdere missie-scenes geactiveerd. Hierdoor is het systeem duidelijk opgebouwd als een scene-gedreven game/applicatie.

Laag	Beschrijving	Voorbeelden
Scene/UI-laag	Verwerkt knoppen, navigatie en scherminteractie	LoginScript, MenuSceneManager, PollSceneManager
Applicatielaag	Beheert gebruikersflow per scene	DagboekSceneManager, AfspraakManager, Avatarcostumizer
Integratielaag	Vertaling van game-acties naar API-calls	UserApiClient, PatientApiClient, DagboekApiClient, AfspraakApiClient
Transportlaag	HTTP requests, tokenbeheer en foutafhandeling	WebClient

De API-base URL komt op meerdere plekken terug in scènes en prefabs. Functioneel werkt dit, maar voor beheer en onderhoud is een centralere configuratie-opzet wenselijk.

4. Backendanalyse: .NET Web API

De webapi-repo gebruikt **.NET 9**, **ASP.NET Core Web API**, **Dapper** en **Avans.Identity.DapperOrm**. In de solution zitten twee projecten: *ZorgWebApi* voor de runtime-applicatie en *UnitTestenZorgApp* voor de tests. In `Program.cs` worden controllers, Swagger/OpenAPI, repositories, services, Dapper stores en Identity API endpoints geregistreerd.

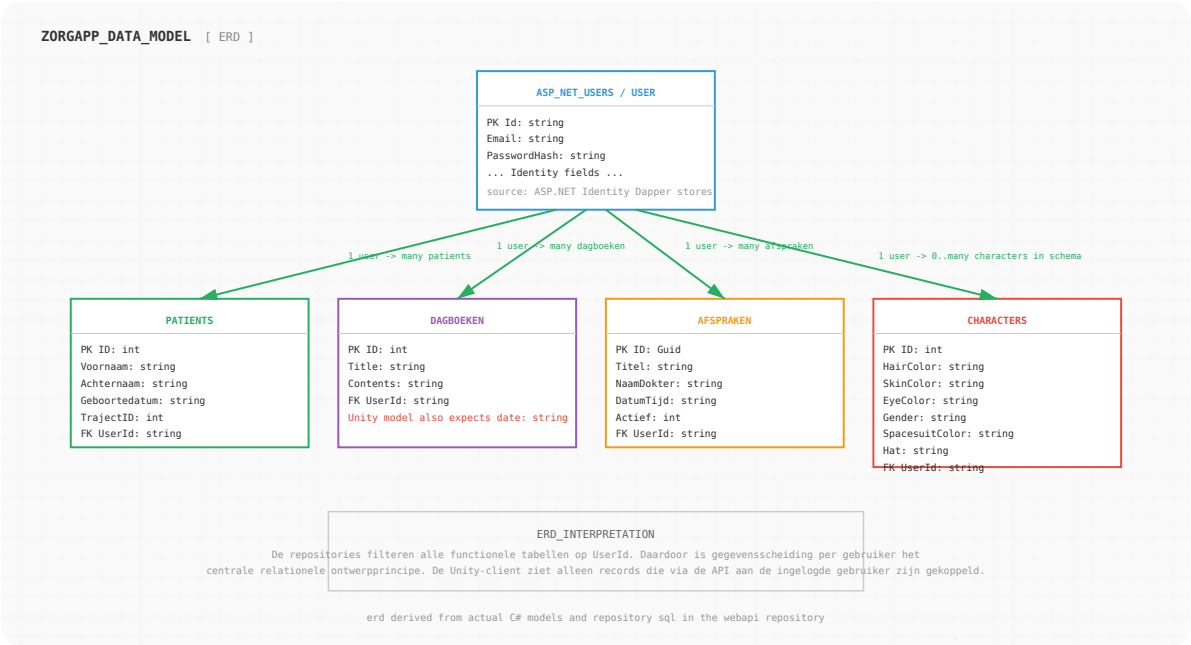


Figuur 2 — UML class diagram van kernklassen, interfaces en modellen in de backend. Inclusief types, methoden en afhankelijkheden.

5. Relatie tussen API en Microsoft SQL Server

De API maakt in `Program.cs` een `SqlConnection` aan op basis van een connection string uit configuratie. Vervolgens gebruiken de repositories Dapper-query's voor CRUD-operaties. Voorbeelden uit de code zijn onder andere `SELECT * FROM Patients WHERE UserId = @UserId`, `SELECT * FROM Dagboeken WHERE UserId = @UserId`, `SELECT * FROM Afspraken WHERE UserId = @UserId` en `SELECT * FROM Characters WHERE UserId = @UserId`.

Daardoor is de database logisch opgezet rondom de gebruiker. Niet de Unity-client, maar de backend bepaalt welke gegevens een gebruiker mag zien. Dit is een belangrijk ontwerpvoordeel op het gebied van beveiliging en onderhoudbaarheid.



Figuur 3 — ERD van het functionele datamodel. Alle hoofdtabellen koppelen terug naar de gebruiker via `UserId`.

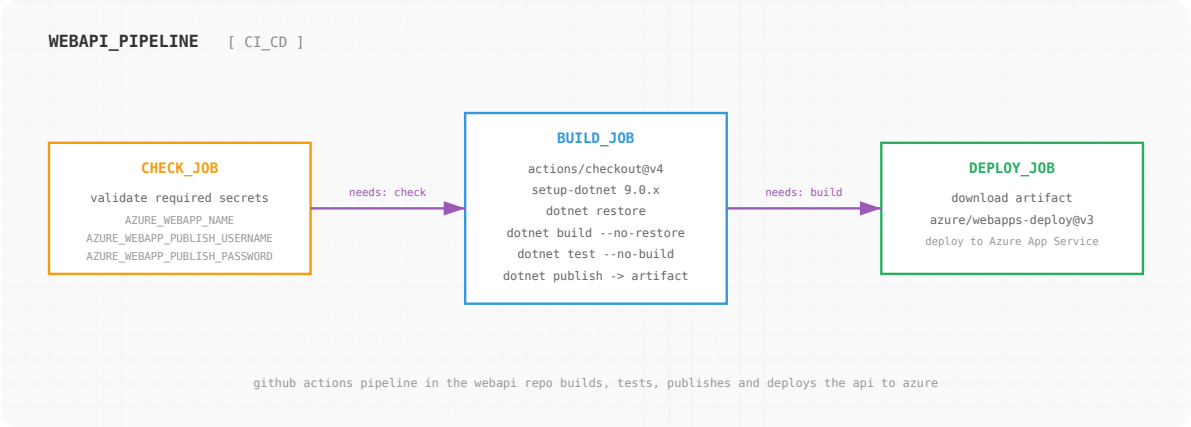
6. Belangrijkste runtime-interacties

De sterkste functionele flow in het systeem is de keten **registreren of inloggen → token ontvangen → token opslaan → Bearer-token meesturen → user-scoped databasequery**. Dit patroon komt terug bij het ophalen van patiënten, dagboekitems en afspraken.

Stap	Wat gebeurt er	Componenten
1	Gebruiker logt in via Unity	LoginScript → UserApiClient
2	Unity stuurt POST naar /account/login	WebClient → Web API
3	API valideert via ASP.NET Identity	Identity + SQL Server
4	Unity ontvangt token en slaat dit op	PlayerPrefs
5	Vervolgrequests bevatten Bearer-token	WebClient
6	API filtert records op UserId	controllers + repositories + SQL

7. CI/CD-pipeline

In de webapi-repo is een GitHub Actions workflow aanwezig: `.github/workflows/github_workflow_simplified (2).yml`. Deze pipeline wordt gestart op pushes naar `main`, `deployment` en `master`, en kan ook handmatig gestart worden via `workflow_dispatch`.



Figuur 4 — CI/CD-flow voor de Web API: secrets-check, build/test/publish en deployment naar Azure App Service.

De pipeline bestaat uit drie fasen:

- **Check:** controleert of verplichte Azure-secrets aanwezig zijn.
- **Build:** voert dotnet restore, dotnet build, dotnet test en dotnet publish uit.
- **Deploy:** downloadt het artifact en probeert naar Azure App Service te deployen via azure/webapps-deploy.

> **Pipeline-opbouw:** de workflow bestaat uit een controlefase voor secrets, een build- en testfase en een deployfase richting Azure App Service. Daarmee ondersteunt de pipeline een consistente releaseflow voor de backend.

8. Sterke punten, risico’s en alternatieven

Sterke punten

- Duidelijke client-serverarchitectuur
- Goede basis van gelaagde backendarchitectuur
- User-scoped filtering via UserId
- Snelle data-access via Dapper
- Aanwezige testprojecten en CI/CD-pipeline

Risico’s

- Tokenopslag in PlayerPrefs is beperkt veilig
- Frontend en backend lijken niet volledig synchroon op alle endpoints
- Configuratie van base URLs is versnipperd
- Niet alle businessregels zitten in services
- Handgeschreven SQL blijft foutgevoeliger

9. Koppeling aan de rubric

Rubric-eis	Hoe deze documentatie eraan voldoet
Overzicht van componenten en interacties	De frontend, backend, SQL Server en pipeline zijn expliciet beschreven en visueel gemaakt.
Architectuurprincipes toepassen	Client-server, gelaagde architectuur en separation of concerns zijn inhoudelijk uitgelegd en gekoppeld aan echte code.
Visuele weergave van componenten	Architectuurdiagram, UML-diagram, ERD en CI/CD-diagram zijn toegevoegd.
Ontwerpkeuzes en alternatieven	Dapper, Identity, Unity, tokenopslag en configuratiekeuzes zijn uitgelegd met alternatieven.
Schaalbaarheid, onderhoudbaarheid, performance	Keuzes zijn gekoppeld aan impact op beheer, veiligheid en verdere uitbreiding.

Kwaliteitsniveau van het ontwerp: de documentatie beschrijft niet alleen welke onderdelen er zijn, maar ook hoe deze samenwerken en waarom de gekozen architectuur logisch is voor onderhoudbaarheid, schaalbaarheid, security en performance.

10. Conclusie

De ZorgApp 1.3 gebruikt een architectuur die technisch logisch en goed verdedigbaar is voor dit project. De Unity-client verzorgt de interactieve gebruikerservaring, de .NET Web API centraliseert authenticatie en businesslogica, en SQL Server bewaart zowel identity-data als functionele appdata. Dankzij de aanwezigheid van repositories, interfaces, testprojecten en CI/CD is er meer aanwezig dan alleen een werkend prototype: er is een duidelijke basis voor verdere professionele doorontwikkeling.

De relatie tussen componenten is duidelijk terug te zien in de implementatie: Unity gebruikt API-clients en een centrale WebClient, de API gebruikt Identity en Dapper, en de databasekoppeling draait volledig op server-side filtering met `UserId`. Daardoor is het software-ontwerp compleet genoeg om de applicatie te realiseren en tegelijk duidelijk genoeg gedocumenteerd voor verdere uitbreiding en onderhoud.

4
diagrammen opgenomen

2
codebases onderzocht

1
complete inlever-PDF